

IoT Challenge 3

Node-RED + ThingSpeak

Mohammad Javad Zandiyeh

Person Code: 11044962

May 11, 2026

1. Overview

The goal of the challenge is to build a single Node-RED flow that simulates a ZigBee-aware MQTT pipeline driven by a pre-recorded sniffer trace (**challenge3.csv**). The flow:

1. Publishes a random *id* every second on **challenge3/id_generator** and logs every published message to **id_log.csv**.
2. Subscribes to the same topic, computes $N = \text{id} \bmod 5218$, and looks up packet No. = N in the CSV.
3. Depending on the packet type, it either republishes a ZBEE_ZCL command on a per-device topic (rate-limited to 10 msg/min), accumulates ZigBee link costs, or simply ignores the message.
4. After exactly 200 received id-messages the flow finalises: **outgoing_cost.csv** is written and the costs from the smallest source address are pushed to a public ThingSpeak channel at one update every 20s.
5. RMS Voltage, RMS Current and Active Power values found in ZBEE_ZCL payloads are also written to **filtered_elems.csv** and plotted live in two Node-RED dashboard charts.

The local infrastructure runs entirely in Docker: Mosquitto on port 1884 and Node-RED with a shared volume containing **challenge3.csv** and the produced CSVs.

2. The Node-RED Flow

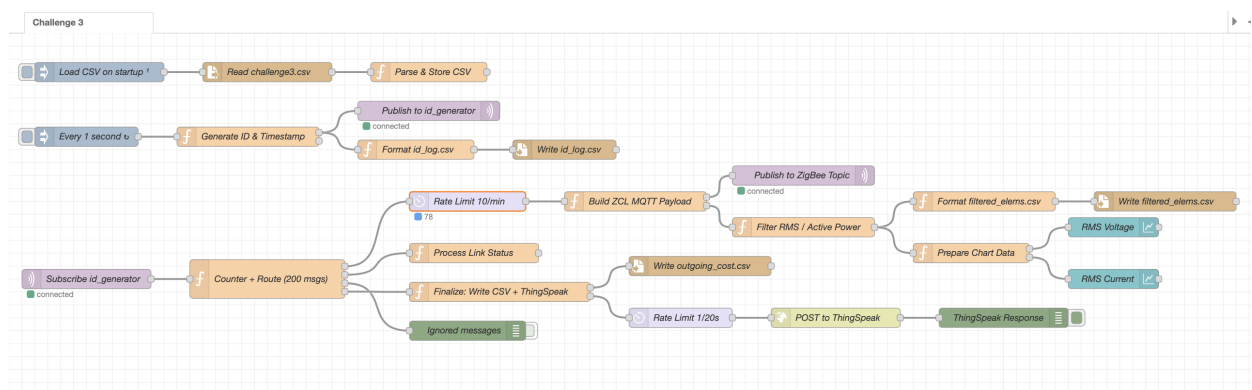


Figure 1: Complete Node-RED flow “Challenge 3”. Three independent branches share a single tab: *Init* (loads **challenge3.csv** once at startup), *Publisher* (top, emits one MQTT id-message per second) and *Subscriber + Router* (bottom, receives the messages, counts up to 200, and routes them to the ZCL, Link-Status, Ignore or Finalize sub-paths).

2.1 Init branch: load challenge3.csv once

- **Load CSV on startup** – inject node fired once, 0.5s after deploy. Kicks off the loader chain.
- **Read challenge3.csv** – file in node that reads the whole CSV (mounted at /challenge3/challenge3.csv inside the container) as a UTF-8 string.
- **Parse & Store CSV** – function node with a small CSV parser that handles double-quoted, comma-containing fields. It builds an object `{packetNumber → row}`, stores it in the global context as `csvData`, and initialises the counters used by the rest of the flow (`msgCount`, `logCounter`, `filteredCounter`, `linkCosts`, `done`).

2.2 Publisher branch: challenge3/id_generator

- **Every 1 second** – inject configured with `repeat=1s`, so a tick is emitted once per second.
- **Generate ID & Timestamp** – function node that produces `id ∈ [0, 30000]` and the current UNIX timestamp, and serialises them as JSON. The output is fan-out to two nodes.
- **Publish to id_generator** – mqtt out node targeting topic `challenge3/id_generator` on the local broker (Mosquitto at `mosquitto:1884`).
- **Format id_log.csv** – function that increments an incremental row number and produces a `No.,ID,TIMESTAMP` line. The header is prepended on the first row.
- **Write id_log.csv** – file node in *append* mode that writes the formatted line to /challenge3/id_log.csv.

2.3 Subscriber + Router branch (200-message counter)

- **Subscribe id_generator** – mqtt in node subscribed to `challenge3/id_generator` on the same local broker.
- **Counter + Route (200 msgs)** – the heart of the flow. This function node has **four outputs**:
 1. Bails out early if the global flag `done` is set.
 2. Increments `msgCount`; the 200th message sets `done = true` and emits a one-shot `finalize` message on output 4.
 3. Computes $N = \text{id} \bmod 5218$ and looks up the corresponding row in the cached CSV.
 4. Routes the message according to its packet type:
 - Output 1 – the row contains `Layer ZBEE_ZCL` → ZCL path.
 - Output 2 – the row's Packet Type is `Link Status (0x08)` → Link Status path.
 - Output 3 – anything else (including missing rows for the random N) → Ignore.
 - Output 4 – one-shot finalize trigger when the 200-msg cap is reached.
- **Ignored messages** – debug sink for the third output; ignored messages still contribute to the 200-message limit, as required.

2.4 ZCL path: per-device MQTT republish + RMS extraction

- **Rate Limit 10/min** – delay node configured in *rate* mode at 10 messages per minute, as required by the challenge.
- **Build ZCL MQTT Payload** – function that builds the required JSON payload (`timestamp`, `id`, `wpan.src`, `wpan.dst`, `zbee.src`, `zbee.dst`, `topic = ZigBee/<DevName>`, `payload = CommandString`). Two outputs: the first is the MQTT message to publish; the second is the original `msg` carried over for downstream RMS filtering.
- **Publish to ZigBee Topic** – mqtt out that publishes on the topic set by the function node above (`ZigBee/<Device Name ZigBee Source>`).

2.5 RMS-filter sub-path: chart + filtered_elems.csv

- **Filter RMS / Active Power** – function that parses the Command String (it is a Python-style dict, so it is converted to JSON first), keeps only rows whose **Attribute** contains *Active Power*, *RMS Current* or *RMS Voltage*, and matches each attribute with the **Status**, **Data Type** and the correctly-indexed numeric value (e.g. `Int16[0]` or `UInt16[1]`) according to the rule “*elements are matched by position in their respective lists*”. Read-Attribute requests (no **Status**) are skipped.
- **Format filtered_elems.csv** – function that appends a numbered line per extracted attribute in the format `No.,Timestamp,Sequence Number,Attribute,Status,Data Type,Data Value`; the header is written only on the first row (`filteredCounter == 0`).
- **Write filtered_elems.csv** – file node in append mode that writes the line to disk.
- **Prepare Chart Data** – function with two outputs that emits a `{payload: value}` message on output 1 for every RMS Voltage and on output 2 for every RMS Current.
- **RMS Voltage / RMS Current** – `ui_chart` nodes drawing the two line charts on the Node-RED dashboard.

2.6 Link-Status path: accumulate outgoing costs

- **Process Link Status** – function that parses the Command String of Link-Status (0x08) packets, iterates over the **Links** array, and for each (`zbee.src`, `lnk.Address`) pair stores or overwrites `linkCosts[key] = {source, destination, cost}` in the global context, so that later Link-Status packets from the same source update the previously stored cost.

2.7 Finalize path: CSV + ThingSpeak

- **Finalize: Write CSV + ThingSpeak** – function fired exactly once at the 200th message. It (i) sorts the accumulated link costs by source then destination treating both as hexadecimal, (ii) renders the `No.,Source,Destination,Cost` CSV, (iii) finds the *smallest* ZigBee source address and the destinations associated to it, sorted ascending in hex, and (iv) emits one ThingSpeak HTTP-request message per destination, with the outgoing cost packed into `field1`.
- **Write outgoing_cost.csv** – file node (overwrite mode) that writes the CSV produced above.
- **Rate Limit 1/20s** – delay node configured for one message every 20 seconds, throttling the ThingSpeak updates as required.
- **POST to ThingSpeak** – http request POSTing a JSON body `{api_key: <write key>, field1: <cost>}` to <https://api.thingspeak.com/update.json>.
- **ThingSpeak Response** – debug that prints the entry ID returned by ThingSpeak so deliveries can be inspected.

3. Produced CSV files

All three CSVs are produced inside the Node-RED working directory (mounted to `./`). Their first lines are shown below to confirm the required formats.

id_log.csv (header + first rows)

```
No.,ID,TIMESTAMP
1,8718,1778525095
2,21298,1778525096
3,13820,1778525097
```

```
4,18462,1778525098
5,12991,1778525099
...
```

filtered_elems.csv (all rows)

```
No.,Timestamp,Sequence Number, Attribute,Status,Data Type,Data Value
1,1778525095,70,Active Power,Success,16-Bit Signed Integer,0
2,1778525095,70,RMS Current,Success,16-Bit Unsigned Integer,0
3,1778525095,70,RMS Voltage,Success,16-Bit Unsigned Integer,219
4,1778525197,68,Active Power,Success,16-Bit Signed Integer,0
5,1778525197,68,RMS Current,Success,16-Bit Unsigned Integer,0
6,1778525197,68,RMS Voltage,Success,16-Bit Unsigned Integer,220
7,1778525209,197,Active Power,Success,16-Bit Signed Integer,0
8,1778525209,197,RMS Current,Success,16-Bit Unsigned Integer,0
9,1778525209,197,RMS Voltage,Success,16-Bit Unsigned Integer,222
...
21,1778525401,25,RMS Voltage,Success,16-Bit Unsigned Integer,220
```

outgoing_cost.csv (header + excerpt)

```
No.,Source,Destination,Cost
1,0x0000,0x0112,7
2,0x0000,0x1cd8,1
3,0x0000,0x1e15,3
4,0x0000,0x3d95,3
5,0x0000,0x4e11,3
6,0x0000,0xa706,3
7,0x0000,0xe1a6,1
8,0x0000,0xec7f,1
9,0x0112,0x0000,1
...
70,0xec7f,0xe1a6,1
```

The smallest source address in the table is 0x0000, so the costs uploaded to ThingSpeak come from rows 1–8 in destination-hex order (0x0112, 0x1cd8, 0x1e15, 0x3d95, 0x4e11, 0xa706, 0xe1a6, 0xec7f) with values {7, 1, 3, 3, 3, 3, 1, 1}.

4. ThingSpeak channel

The outgoing costs of the smallest ZigBee source are pushed (one update every 20 seconds, in destination-hex order) to a **public** ThingSpeak channel:

Channel ID: 3378529
Name: Polimi IOT Outgoing Cost
Field 1: outgoing cost
URL: <https://thingspeak.mathworks.com/channels/3378529>
Feed CSV: <https://thingspeak.mathworks.com/channels/3378529/feed.csv>
Field-1 CSV: <https://thingspeak.mathworks.com/channels/3378529/field/1.csv>

5. Notes on the implementation

- **State.** All cross-message state lives in the Node-RED global context (csvData, msgCount, logCounter, filteredCounter, linkCosts, done), so no extra Node-RED context store is required.

- **200-message gate.** The counter is incremented for *every* subscribed message (ZCL, Link Status, ignored or unmatched), as the specification requires. Once `msgCount` $\geq$ 200 the `done` flag short-circuits every further message at the top of the router, while the one-shot `finalize` trigger drives the CSV write and the ThingSpeak burst.
- **RMS value matching.** The Command String is a Python-style dict; a small `py2json` helper rewrites single quotes and Python literals so the payload can be parsed with `JSON.parse`. The correct value for each attribute is then picked from `Int16`, `UInt16`, etc., using a per-type running index, which faithfully implements the “matched by position” rule of the specification.
- **Rate limits.** Both rate-limiters use the `delay` node in *rate* mode (no dropping): 10/minute on the ZCL republish and 1/20 s on the ThingSpeak POST.